Networks1 (10636454) –Project Report

ChatLite — Minimal Real-Time Chat System using Java Sockets

Student #1:
Noora Qaradeh 12325548

Student #2:
Malak Fatayer 12325392

Instructor: Dr. Saed Tarapiah

Introduction

ChatLite is a minimal real-time chat system developed to demonstrate basic client-server communication using Java sockets. The project shows how multiple clients can connect to a central server, join chat rooms, exchange public and private messages, and manage user sessions. It provides a hands-on example of how real-time communication systems work in a clear and educational way.

System Architecture

ChatLite consists of two main components: a Server Application and a Client Application. The overall architecture is based on a single communication channel:

- TCP Channel – Used for login, joining rooms, sending public messages, private messaging, retrieving user lists, and status updates.

Each client establishes a persistent TCP connection with the server. The server manages connected users, chat rooms, and message broadcasting, while the client provides a graphical interface for interaction.

System Overview

ChatLite follows a distributed architecture composed of a central server and multiple JavaFX-based clients. The server is responsible for authentication, managing chat rooms, routing public and private messages, logging system events, and tracking user presence. Each client provides a graphical interface for logging in, joining rooms, sending messages, viewing online users, and managing private conversations.

Technical Implementation

The system is fully implemented in Java using TCP sockets for reliable communication between the server and each client. TCP ensures that login, room joining, message delivery, and status updates are transmitted consistently and in order.

Each client connects to the server through a persistent TCP channel. The server uses multi-threading (via Threads or ExecutorService) to handle multiple clients concurrently, ensuring real-time responsiveness.

The client application is built with JavaFX, providing a modern GUI that includes chat rooms, user lists, message tables, private messaging panels, and search functionality.

Network Setup using ZeroTier

To connect multiple laptops on different networks, we used ZeroTier to create a virtual LAN.

Each device joined the network using a unique ID and received a virtual IP address, allowing all machines to communicate as if they were on the same local network. The server was hosted on one laptop, and clients connected using the server's ZeroTier IP.

ZeroTier enabled secure, peer-to-peer communication and simplified network configuration, allowing us to test the ChatLite system in a realistic multi-client environment without requiring a shared physical network.

Server Responsibilities (ChatLite Server)

Connection Management:
Listens on TCP port 5000 and accepts incoming client connections. Maintains persistent sessions for each connected user.

Multi-threading:
Handles multiple clients concurrently using threads, ensuring real-time responsiveness and smooth communication.

Authentication & User Management:
Provides login functionality, verifies usernames and passwords against entries stored in a text file (`users.txt`), and allows administrators to add, delete, or reset user accounts.

Active Sessions Tracking:
Displays connected users along with their IP addresses, and monitors their online status.

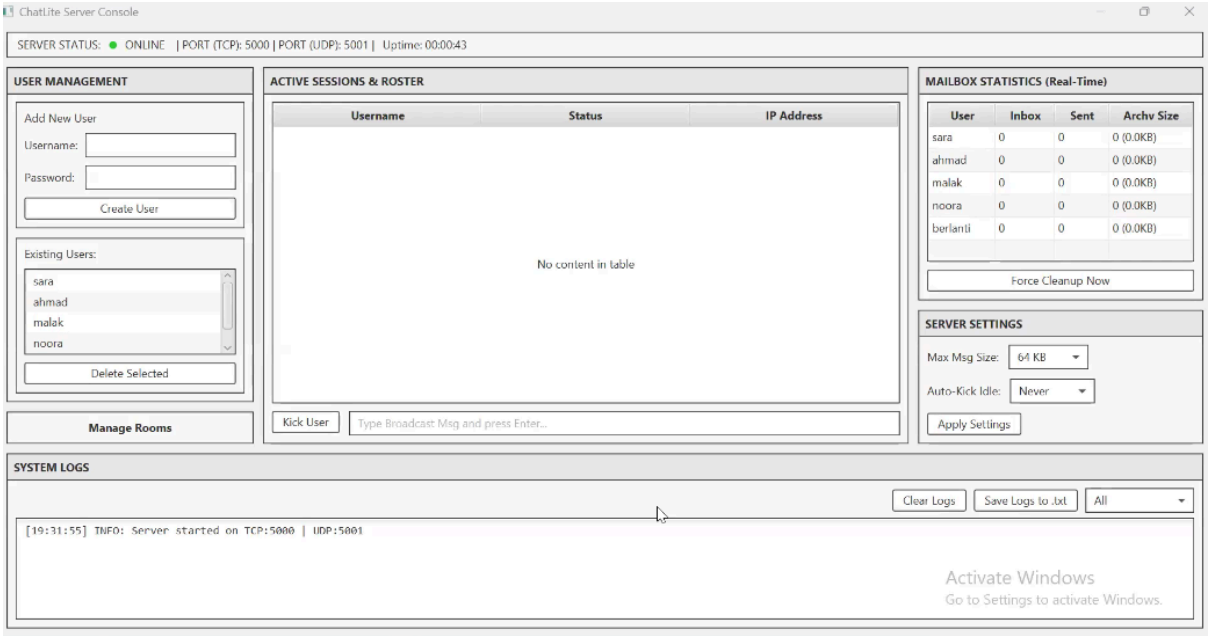
Message Routing:
Manages chat rooms and routes messages (public and private) to the correct recipients.

Mailbox Statistics:
Tracks inbox, sent, and archived messages for each user, providing real-time statistics.

Broadcast Capability:
Allows administrators to send system-wide announcements to all connected clients.

System Logging:
Records events such as server start, user login/logout, and errors in a live stream log. Supports saving logs to text files and filtering.

Configuration Options:
Enables customization of server parameters (e.g., maximum message size) to control system behavior.



Client Responsibilities (ChatLite Client)

Connection Management:
Connects to the server via TCP (default port 5000) and maintains a persistent session. Displays connection status and uptime to the user.

Authentication:
Provides a login dialog where the user enters server IP, username, and password. Credentials are verified by the server against the `users.txt` file.

Graphical User Interface (GUI):
Built with **JavaFX**, offering a modern interface that includes:

- Chat Rooms list with Join/Leave options.
- Online Users panel showing connected users with status indicators.
- Chat Messages table with username, message, and timestamp.
- Message input field and Send button.
- Private messaging panels for direct communication.

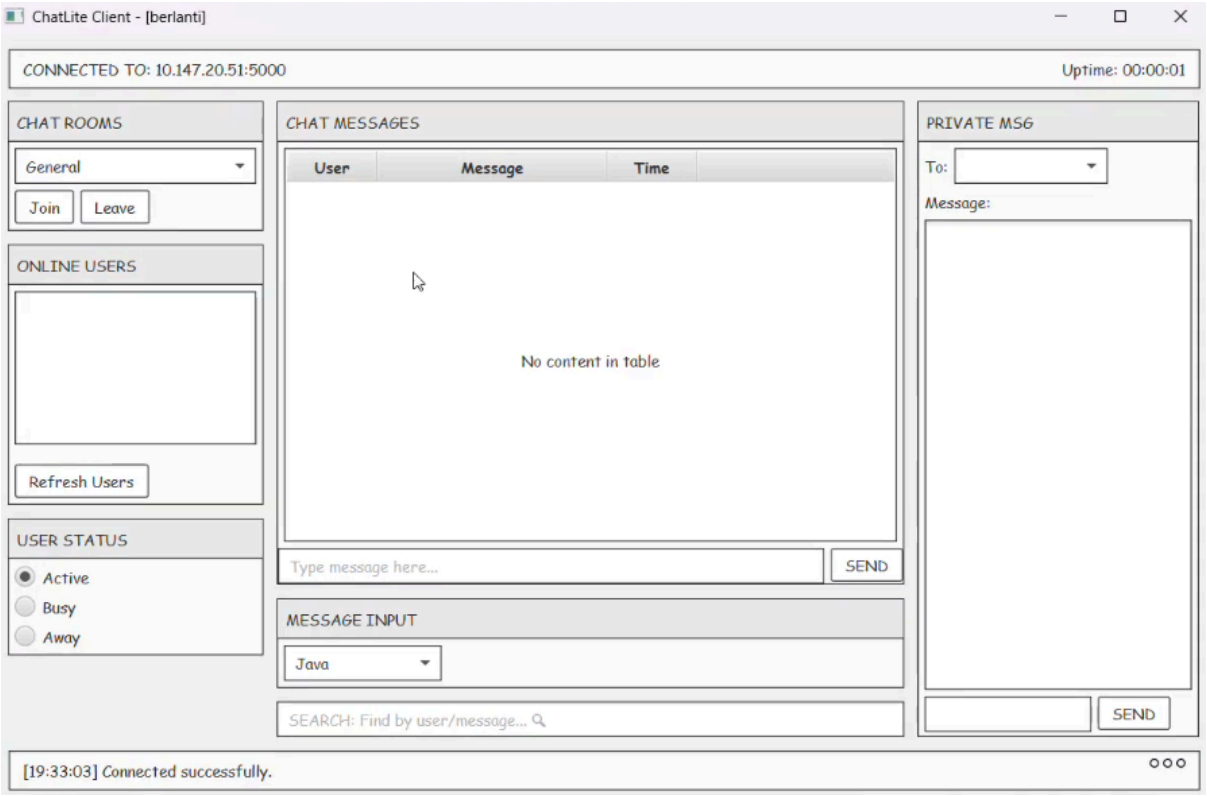
- Search box for filtering messages by user or text.
- Status bar showing logs and connection feedback.

Room Management:
Allows users to join or leave rooms (e.g., General, Networks, Java) and participate in group conversations.

Messaging:
Enables sending public messages to rooms and private messages to individual users. Messages are displayed in real-time with sender and timestamp.

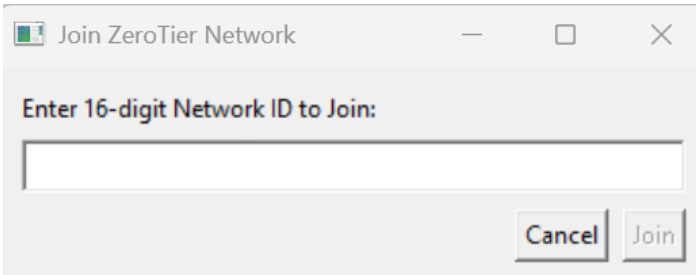
User Presence & Status:
Provides options to set status (Active, Busy, Away).

Search & Filtering:
Offers a search box to quickly locate specific messages or conversations.

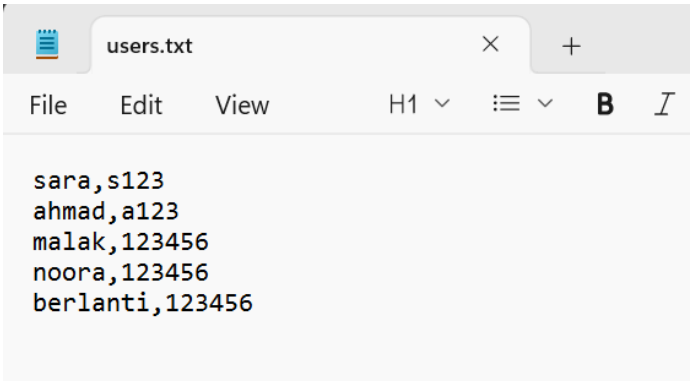


Testing & Demonstration:

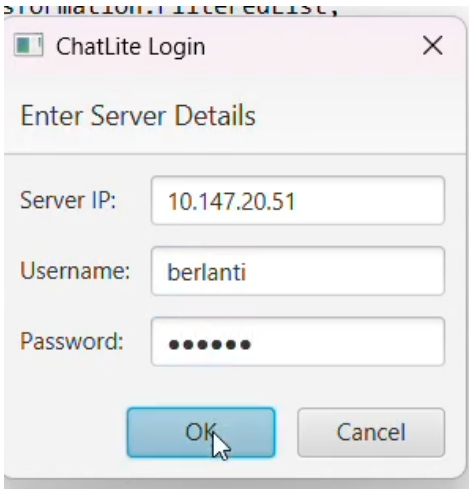
• We used 3 different laptops (1 as server and 2 as users) to test the application, each of them joined the virtual network that was created using ZeroTier, and has a unique ID on the network.

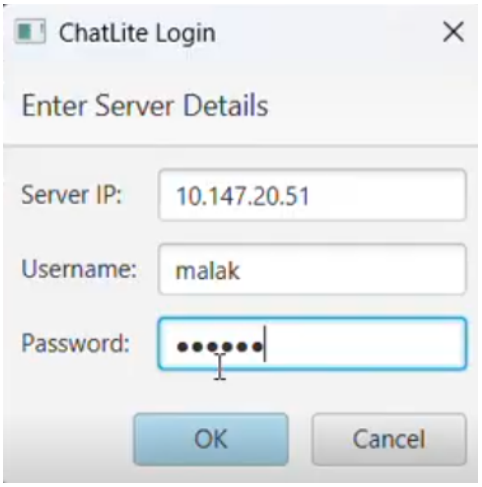


• After launching the server, users were added and confirmed inside users.txt.



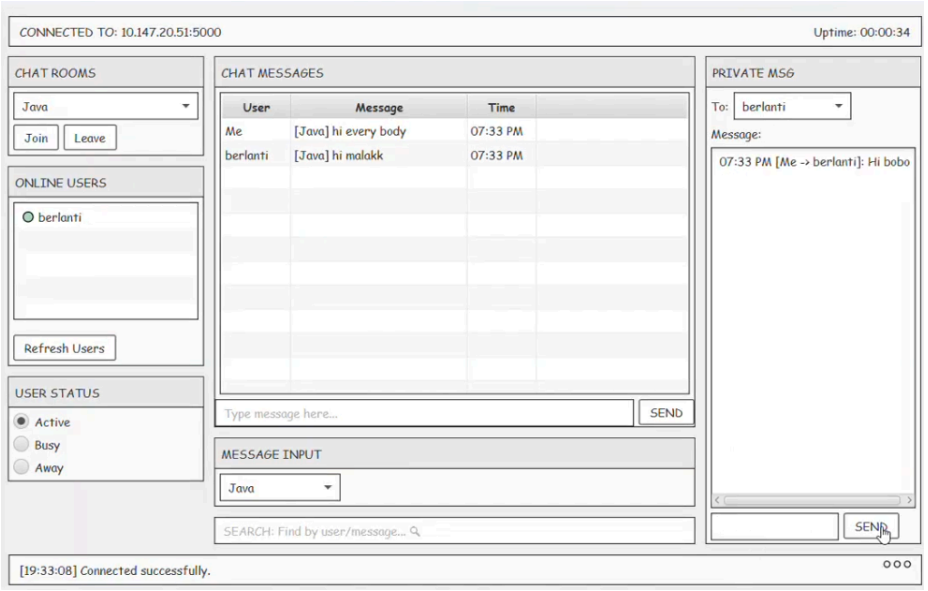
• Each user logged in from different laptops



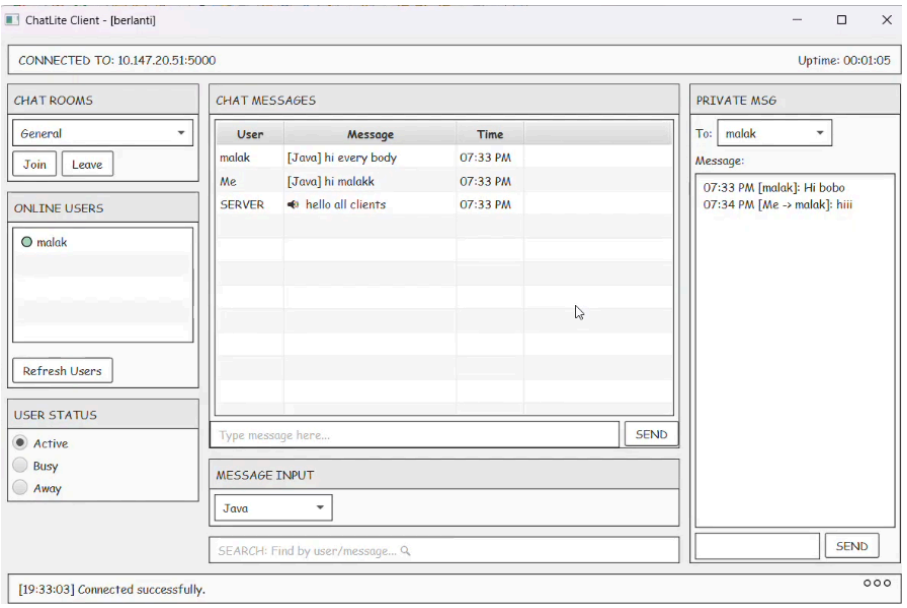


• Each user can join rooms and send messages for all members in the room, or send private message to any active user

Malak's side:

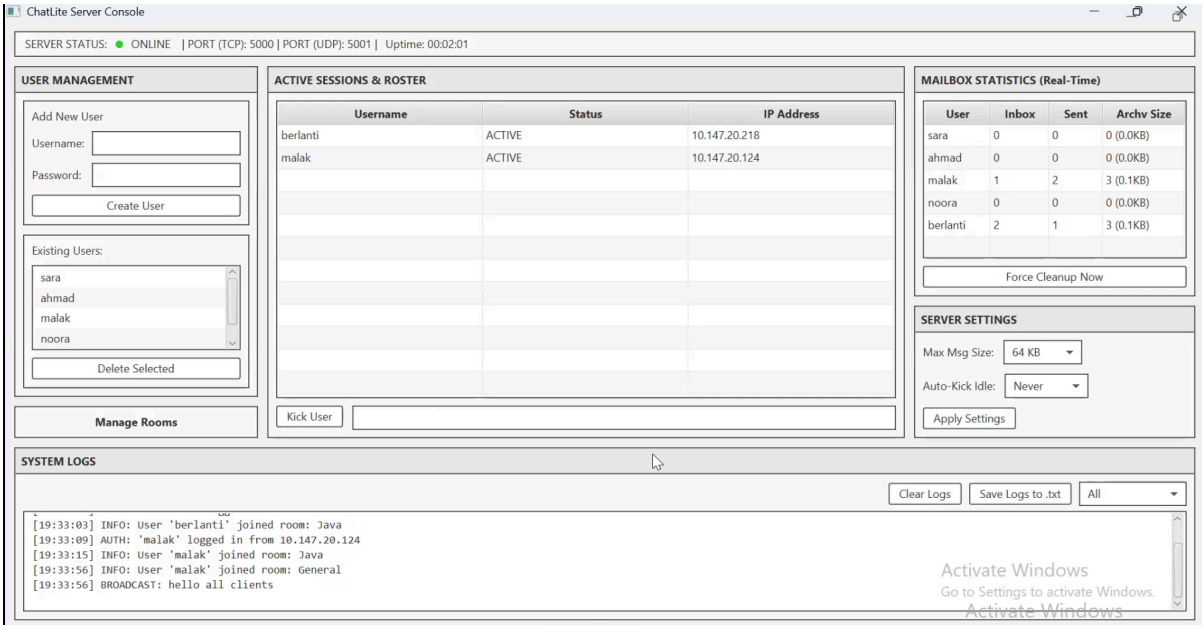


berlanti's side:

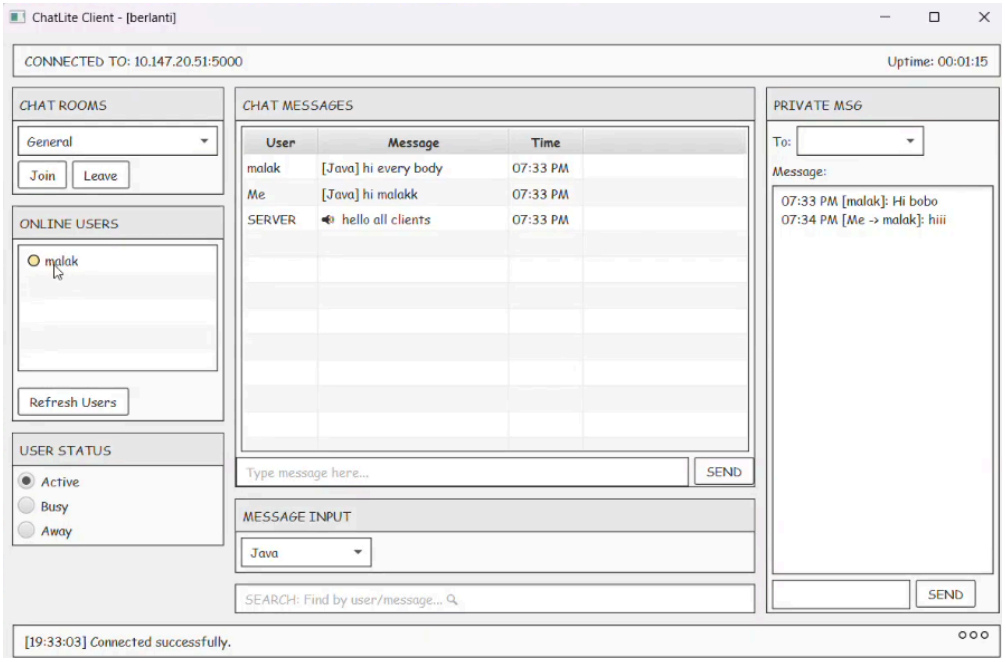


<u>note here that server send a broadcast msg to all users</u>

server’s side (notice mailbox statistics && system logs):

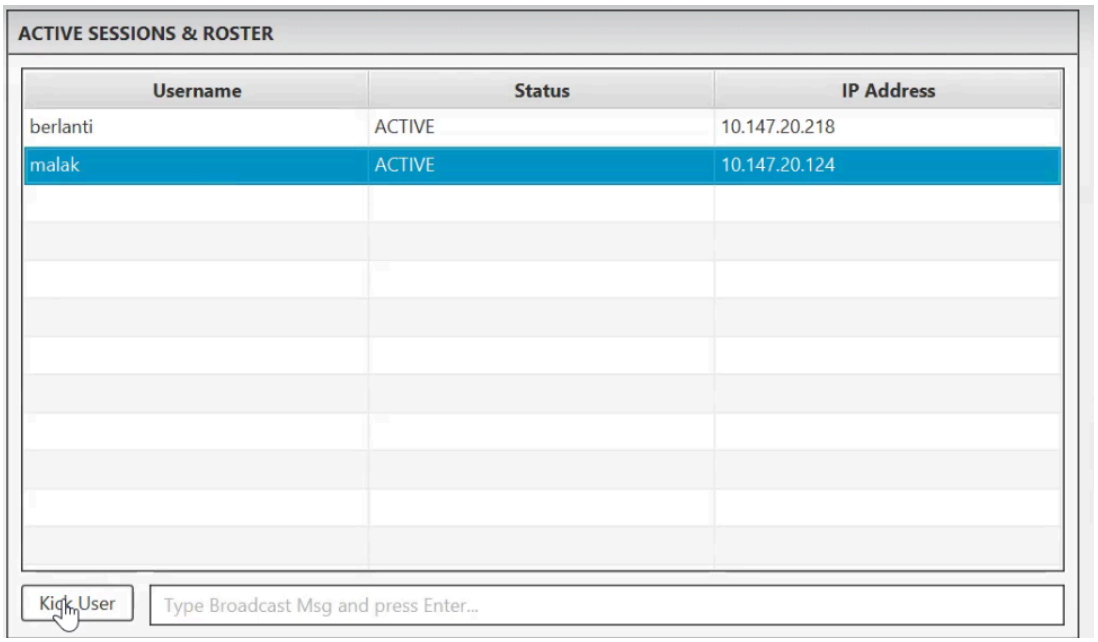


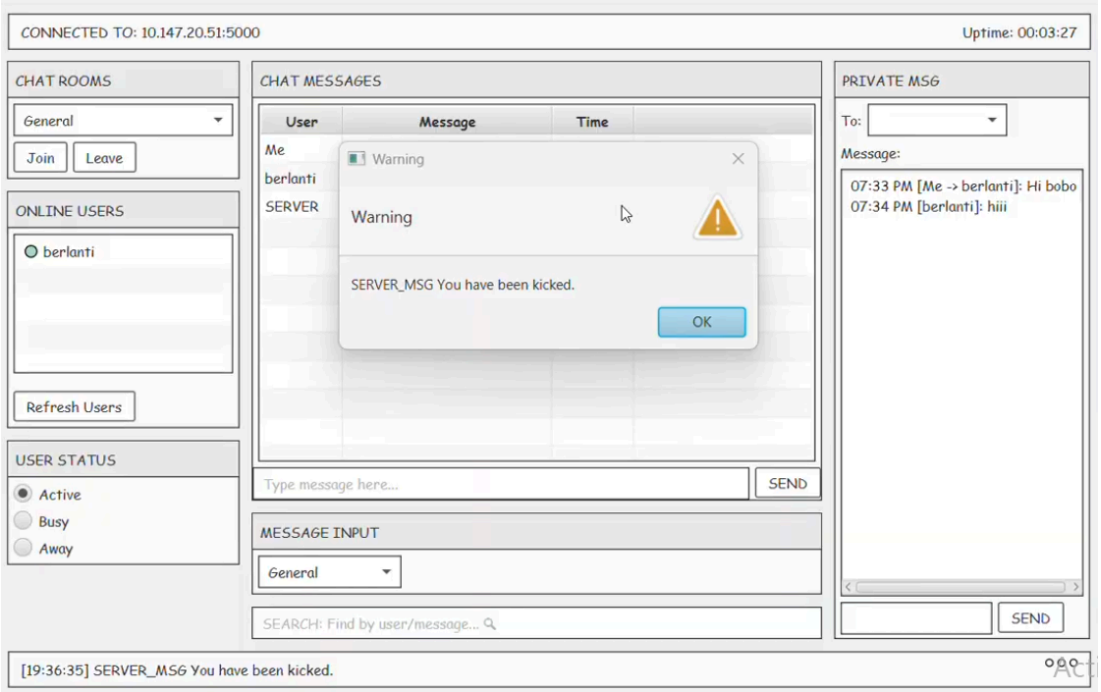
• users can change their status using user status area



<u>note: here malak changed her status to busy and it appeared in berlinti’s side</u>

• Server also can kick users, as server kick malak here





<u>important note: there are also more features in server that we show in the server's video</u>

Limitations

- No attachment or file transfer support.
- No encryption or secure channels (plain TCP only).
- Centralized architecture (single server, no load balancing).
- Limited mailbox statistics (basic counts only, no advanced analytics).
- GUI is desktop-only (JavaFX), no mobile client.

Future Improvements

- Add support for file sharing and attachments.
- Implement TLS encryption for secure communication.
- Introduce typing indicators and message read receipts.
- Add push notifications for mobile devices.
- Enhance mailbox statistics with detailed analytics.
- Support distributed or multi-server architecture for scalability.

Lessons Learned

- Gained solid understanding of **TCP client/server programming** using Java sockets.
- Learned how to design and parse a simple text-based communication protocol.
- Practiced implementing **multi-threaded servers** to handle concurrent clients.
- Built a real-time communication system with **JavaFX GUI**.
- Improved skills in event-driven programming and user interface design.
- Experienced testing across multiple clients and chat rooms.

Conclusion

ChatLite proved to be a functional and stable real-time chat system. It successfully demonstrated multi-client communication, chat rooms, private messaging, and server-side user management. Future enhancements may include encrypted communication, file attachments, typing indicators, and mobile support to make the system more robust and closer to modern messaging applications.